



# DATASTRUCTURES AND ALGORITHMS

## 23CSE203

## LAB MANUAL

|                  |                      |
|------------------|----------------------|
| SUBMITTED BY     |                      |
| NAME             | B.Manaswini          |
| ROLL NO          | AV.SC.U4CSE25102     |
| YEAR/SEM/SECTION | 2ndYEAR/ SEM-3/CSE-B |
| CREDITS          | 05                   |

|              |                           |
|--------------|---------------------------|
| SUBMITTED TO | DR. RAJ KUMAR BATCHU      |
| DESIGNATION  | ASST.PROFFESSOR(SL Grade) |
| DEPARTMENT   | SCHOOL OF COMPUTING       |

# INDEX

| S.N<br>O | TITLE                                                                                                                                                                                                                                                            | PAGE<br>NO | REMARKS |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|---------|
| 1        | WEEK 1 : a. Countdown from n to 1, then print " <b>Launch</b> "<br>b. Calculate a power $p^n$ recursively<br>c. Search for an employee ID in a list recursively<br>d. Calculate factorial $n!$ recursively<br>e. Print the first n Fibonacci numbers recursively |            |         |
| 2        | WEEK 2 : 1 . Write a python program to implement linear<br><br>search<br><br>2. Write a python program to implement binary<br><br>search with and without sorted array as an input                                                                               |            |         |
| 3        | WEEK 3 : write programs for the given sorts<br><br>1. Bubble sort<br><br>2. Insertion sort<br><br>3. Selection sort                                                                                                                                              |            |         |

# INDEX

| S.N<br>O | TITLE    | PAGE<br>NO | REMARKS |
|----------|----------|------------|---------|
| 4.       | WEEK 4 : |            |         |
| 5        | WEEK 5:  |            |         |

# INDEX

| S.NO | TITLE    | PAGE NO | REMARKS |
|------|----------|---------|---------|
| 6.   | WEEK 6:  |         |         |
| 7    | WEEK 7 : |         |         |
| 8    | WEEK 8 : |         |         |
| 9.   | WEEK 9 : |         |         |

| S.NO | TITLE     | PAGE NO | REMARKS |
|------|-----------|---------|---------|
| 10.  | WEEK 10:  |         |         |
| 11.  | WEEK 11 : |         |         |

| S.NO | TITLE     | PAGE NO | REMARKS |
|------|-----------|---------|---------|
| 11.  | WEEK 11:  |         |         |
| 12.  | WEEK 12 : |         |         |

# LAB-1

**1.AIM:**A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by “launch”.

## **PROGRAM:**

```
def launch(n):  
  
    for i in range(n,-1,-1):  
  
        if i==0:  
  
            print("launch")  
  
        else:  
  
            print(i)  
  
n=int (input ("Enter countdown number:"))  
  
launch(n)
```

## **OUTPUT:**

```
===== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab1a.py =====  
Enter countdown number:9  
9  
8  
7  
6  
5  
4  
3
```

## **EXPLANATION:**

This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop

## ERROR TABLE:

| S.no | Errors            | Rectification                                                                                             |
|------|-------------------|-----------------------------------------------------------------------------------------------------------|
| 1.   | Indentation Error | Use 3 <b>spaces</b> for each level of indentation.                                                        |
| 2.   | Type Error        | Convert the input to an integer using <code>n = int (input ("Enter no. of seconds to launch: "))</code> . |

**2.AIM:** The banks calculate the growth of an investment using compound interest; the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate  $p^n$  (power), where  $p$ =principal growth factor,  $n$ = number of years.

## PROGRAM:

```
def power (p, n):  
    if n == 0:  
        return 1  
    else:  
        return p * power (p, n - 1)  
  
p = float (input ("Enter principal/base value: "))  
n = int (input ("Enter number of years/powers: "))  
  
result = power (p, n)  
  
print("Power =", result)
```

## OUTPUT:

```
===== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab1b.py =====  
Enter principal/base value: 321  
Enter number of years/powers: 3  
Power = 33076161.0
```

## EXPLANATION:

- The function of power (p, n) calculates  $p^n$  using recursion by multiplying p with the result of power (p, n-1).
- The recursion stops when n becomes 0, returning 1, which is the base case.
- The final result is displayed after all recursive calls are completed.

## ERROR TABLE:

| S.no | Errors                                                                                           | Rectification                                                                          |
|------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| .    |                                                                                                  |                                                                                        |
| 1.   | Type Error                                                                                       | Convert the input to float or int using float(input()) or int(input()).                |
| 2.   | Incorrect output if the recursive call is written as power(p, n + 1) instead of power(p, n - 1). | Change the recursive call to power(p, n - 1) so that n decreases toward the base case. |

**3.AIM:** An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

## PROGRAM:

```
def search (list, key, index):  
    if index >= len(list):  
        return False  
    if list[index] == key:  
        return True  
    return search (lst, key, index + 1)  
  
employees = [101, 102, 103, 104, 105]  
  
key = int (input ("Enter Employee ID to search: "))
```

if search (employees, key, 0):

print (f'Employee ID found at position {key}')

else:

print ("Employee ID Not Found")

## OUTPUT:

```
===== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab1c.py =====  
Enter Employee ID to search: 102  
Employee ID found at position 1
```

## EXPLANATION:

- The recursive function checks each employee ID in the list one by one, starting from the first element.
- If the required ID is found, it returns True; otherwise, it continues searching recursively until the end of the list.
- If the entire list is searched without finding the ID, the function returns False.

## ERROR TABLE:

| S.no | Errors                                                                                | Rectification                                                      |
|------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| .    |                                                                                       |                                                                    |
| 1.   | Name Error if the list name is misspelled (e.g., employee instead of employees).      | Use the correct variable name consistently throughout the program. |
| 2.   | Syntax Error due to a missing colon (:) after an if statement or function definition. | Add the missing colon (:) after if, else, or def.                  |

**4.AIM:** In a logistics company the number of possible ways to arrange n different parcels are calculated using the factorial formula( $n!$ ). develop a recursive program to calculate the factorial of a given number.

## PROGRAM:

```
def factorial(n):  
  
    if n == 0 or n == 1:  
  
        return 1  
  
    else:  
  
        return n * factorial (n - 1)  
  
num = int (input ("Enter a number: "))  
  
if num < 0:  
  
    print ("Factorial does not exist for negative numbers.")  
  
else:  
  
    print ("Factorial =", factorial(num))
```

## OUTPUT:

```
===== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab1d.py =====  
Enter a number: 2  
Factorial = 2
```

## EXPLANATION:

- The function factorial(n) computes the factorial by multiplying the current number with the factorial of the previous number.
- The recursion ends when n becomes 0 or 1, where the factorial value is 1.
- The program also checks for negative inputs since factorial is defined only for non-negative integers.

## ERROR TABLE:

| S.no. | Errors                                                                            | Rectification                                                       |
|-------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------|
| 1.    | IndentationError if the statements inside the function are not properly indented. | Maintain proper indentation (usually 4 spaces) inside the function. |
| 2.    | TypeError if a decimal number (e.g., 5.5) is entered for factorial calculation.   | Enter only a whole number or convert the input using int().         |

**5.AIM:** Write a recursive python program to print the first n terms of the Fibonacci series.

## PROGRAM:

```
def fibonacci(n):  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)  
n = int (input ("Enter the number of terms: "))  
print ("Fibonacci Series:")  
for i in range(n):  
    print (fibonacci(i), end=" ")
```

## OUTPUT:

```
===== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lable.py =====  
Enter the number of terms: 5  
Fibonacci Series:  
0 1 1 2 3  
,
```

## EXPLANATION:

- The function Fibonacci(n) returns the nth Fibonacci number by recursively adding the previous two Fibonacci numbers.
- The base cases return 0 for the first term and 1 for the second term.
- A for loop calls the recursive function repeatedly to print the first n terms of the Fibonacci series.

## ERROR TABLE:

| S.no | Errors                                                                     | Rectification                                          |
|------|----------------------------------------------------------------------------|--------------------------------------------------------|
| 1.   | Recursion Error if the base cases<br>(n == 0 and n == 1) are not included. | Add both base conditions<br>before the recursive call. |
| 2.   | Value Error if the user enters text instead of an<br>integer.              | Enter a valid integer for the<br>number of terms.      |

# LAB-2

**1.AIM:** Write a program to perform linear search on a given input.

## **PROGRAM:**

```
def linear_search(arr,key):  
    for i in range(len(arr)):  
        if arr[i]==key:  
            return i  
    return -1  
  
n = int(input("Enter no. of elements:"))  
  
arr=[]  
  
print("Enter the elements:")  
  
for i in range(n):  
    arr.append(int(input()))  
  
key=int(input("Enter element to search:"))  
  
result=linear_search(arr,key)  
  
if result!=-1:  
    print("Element found at index",result)  
else:  
    print("Element not found.")
```

## **OUTPUT:**

```

===== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab2a.py =====
Enter no. of elements: 3
Enter the elements:
10
20
60
Enter element to search: 60
Element found at index 2

```

## EXPLANATION:

The `linear_search()` function checks each element of the list one by one and compares it with the given key.

- If the key is found, its index is returned; otherwise, the function returns -1.
- The program takes the array and search element as input and displays whether the element is found.

## ERROR TABLE:

| S.no | Errors                                                                                      | Rectification                                                                                                    |
|------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| .    |                                                                                             |                                                                                                                  |
| 1.   | ValueError occurs if a non-integer value is entered when <code>int(input())</code> is used. | Enter only integer values for the number of elements and array elements.                                         |
| 2.   | TypeError may occur if <code>len(arr)</code> is used on a variable that is not a list.      | Make sure <code>arr</code> is initialized as a list using <code>arr = []</code> and elements are appended to it. |

**2.AIM:** Write a Python program to perform binary search on the given input. Verify by taking sorted array and unsorted array as input.

**PROGRAM:**

```
def binary_search(arr,key):

    low=0

    high=len(arr)-1

    while low<=high:

        mid=(low+high)//2

        if arr[mid]==key:

            return mid

        elif arr[mid]<key:

            low=mid+1

        else:

            high=mid-1

    return -1

n = int(input("Enter no. of elements:"))

arr=[]

print("Enter the elements:")

for i in range(n):

    arr.append(int(input()))

if arr==sorted(arr):

    print("\n The input list is already sorted.")

else:

    print("\n The input list is not sorted.")
```

```

print("Sorting the list...")

arr.sort()

print("Sorted list.",arr)

key=int(input("Enter element to search:"))

result=binary_search(arr,key)

if result!=-1:

    print("Element found at index",result)

else:

    print("Element not found.")

```

## OUTPUT:

```

===== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab2b.py =====
Enter no. of elements: 2
Enter the elements:
10
20

The input list is already sorted.
Sorted list: [10, 20]
Enter element to search: 20
Element found at index 1

```

## EXPLANATION:

- The `binary_search()` function repeatedly divides the sorted list into two halves and compares the middle element with the search key.
- If the middle element is smaller than the key, the search continues in the right half; otherwise, it searches the left half.
- The second program verifies whether the input is sorted and sorts it using `arr.sort()` if necessary before performing binary search.

## ERROR TABLE:

| S.no | Errors                                                                                         | Rectification                                                                               |
|------|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| .    |                                                                                                |                                                                                             |
| 1.   | IndexError can occur if mid is calculated or the low/high boundaries are modified incorrectly. | Initialize low = 0, high = len(arr) - 1 and update them as low = mid + 1 or high = mid - 1. |

# Lab-3

**1.AIM:** write programs for the Bubble sort

## **PROGRAM:**

```
#bubble sort

def bubble_sort(arr):

    n = len(arr)

    for i in range (n-1):

        for j in range(n-i-1):

            if arr[j]>arr[j+1]:

                arr[j],arr[j+1] = arr[j+1],arr[j]

    return arr

#taking input

n = int(input("Enter number of elements: "))

arr=[]

print("Enter elements: ")

for i in range(n):

    arr.append(int(input()))

bubble_sort(arr)

print("sorted array: ")

for element in arr:

    print(element, end=" ")
```

## **Output:**

```
===== RESTART: D:/manaswinii-25102/LAB PROGRAMS/lab3a.py =====
Enter number of elements: 4
Enter elements:
10
20
98
65
Sorted array:
10 20 65 98
```

### **Explanation:**

Bubble sort compares two adjacent elements. If the left element is greater than the right element, they are swapped. After each pass, the largest remaining element moves to the end.

### **Error table:**

| S.no | Error                   | Correction                 |
|------|-------------------------|----------------------------|
| 1.   | Used wrong loop range   | Use for j in range(n-i-1): |
| 2.   | Swap condition is wrong | Use if arr[j] > arr[j+1]:  |

## **2.AIM:** writeprograms for the Insertion sort

### **PROGRAM:**

# Insertion Sort

```
def insertion_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(1, n):
```

```
        key = arr[i]
```

```
        j = i - 1
```

```
        while j >= 0 and arr[j] > key:
```

```
            arr[j + 1] = arr[j]
```

```
            j = j - 1
```

```

    arr[j + 1] = key

    return arr

# Taking input
n = int(input("Enter number of elements: "))

arr = []

print("Enter elements: ")

for i in range(n):

    arr.append(int(input()))

insertion_sort(arr)

print("Sorted array: ")

for element in arr:

    print(element, end=" ") same for this

```

## **Output:**

```

===== RESTART: D:/manaswinii-25102/LAB PROGRA
Enter number of elements: 3
Enter elements:
11
76
50
Sorted array:
11 50 76
.

```

## **Explanation:**

Insertion sort builds the sorted array one element at a time.

It takes one element called the **key**, compares it with the elements before it, shifts larger elements to the right, and inserts the key in the correct position.

### **Error table:**

| S.no | Error                                   | Correction                    |
|------|-----------------------------------------|-------------------------------|
| 1.   | key is not stored properly              | Use key = arr[i]              |
| 2.   | arr[j+1] = key placed inside while loop | Place it after the while loop |

**3.AIM:** writeprograms for the Bubble sort

### **PROGRAM:**

# Selection Sort

```
def selection_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n - 1):
```

```
        min_index = i
```

```
        for j in range(i + 1, n):
```

```
            if arr[j] < arr[min_index]:
```

```
                min_index = j
```

```
        arr[i], arr[min_index] = arr[min_index], arr[i]
```

```
    return arr
```

# Taking input

```
n = int(input("Enter number of elements: "))
```

```
arr = []
```

```
print("Enter elements: ")
```

```
for i in range(n):
```

```
arr.append(int(input()))

selection_sort(arr)

print("Sorted array: ")

for element in arr:

    print(element, end=" ")
```

**Explanation:**

Selection sort repeatedly finds the places it in its correct position.     **smallest element** from the unsorted part of the array and

**Error table:**

---

| S.no | Error                            | Correction                      |
|------|----------------------------------|---------------------------------|
| 1.   | min_index is not initialized     | Use min_index = i               |
| 2.   | Swap is placed inside inner loop | Place swap after the inner loop |

# LAB-4

**1.Aim:** Write a python program to implement merge sort where the input elements are taken from the user

## **PROGRAM:**

```
def merge_sort(arr):

    if len(arr) > 1:

        mid = len(arr) // 2

        left = arr[:mid]

        right = arr[mid:]

        merge_sort(left)

        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):

            if left[i] < right[j]:

                arr[k] = left[i]

                i += 1

            else:

                arr[k] = right[j]

                j += 1

            k += 1

        while i < len(left):

            arr[k] = left[i]

            i += 1

            k += 1
```

```
while j < len(right):
```

```
    arr[k] = right[j]
```

```
    j += 1
```

```
    k += 1
```

```
# Input from user
```

```
arr = list(map(int, input("Enter elements separated by spaces: ").split()))
```

```
merge_sort(arr)
```

```
print("Sorted array:", arr)
```

## **Output:**

```
===== RESTART: D:/DSA Programs/programs/Week4.py =====
manaswini-cse25102
Enter elements separated by spaces: 23 75 32 11 8 45 08 53
Sorted array: [8, 8, 11, 23, 32, 45, 53, 75]
```

## **Explanation:**

Merge Sort is a **divide-and-conquer sorting algorithm**. It works in three main steps:

1. Divide – Divide the array into two halves.
2. Conquer – Recursively sort both halves.
3. Merge – Merge the two sorted halves into one sorted array.

## **Error table:**

| S.no | Error                         | Correction                        |
|------|-------------------------------|-----------------------------------|
| 1.   | Wrong base condition          | Use if len(arr) > 1 for splitting |
| 2.   | Wrong midpoint                | Use mid = len(arr)//2             |
| 3.   | Forgetting remaining elements | Append remaining elements         |

**2.Aim:** Write a python program to implement quick sort where the input elements are taken from the user

**Program:**

```
def quick_sort(arr, low, high):

    if low < high:

        pivot = arr[high]

        i = low - 1

        for j in range(low, high):

            if arr[j] < pivot:

                i += 1

                arr[i], arr[j] = arr[j], arr[i]

        arr[i + 1], arr[high] = arr[high], arr[i + 1]

        p = i + 1

        quick_sort(arr, low, p - 1)

        quick_sort(arr, p + 1, high)

# Input from user

arr = list(map(int, input("Enter numbers to sort: ").split()))

quick_sort(arr, 0, len(arr) - 1)

print("Sorted array:", arr)
```

**Output:**

```
= RESTART: C:/Users/CSE Lab1/Desktop/25102/DSA-25102/we
manaswini-cse25102
Enter numbers to sort: 12 34 16 57 85
Sorted array:
[12, 16, 34, 57, 85]
```

## **Explanation:**

Quick Sort is also a **divide-and-conquer sorting algorithm**.

Its main idea is to select an element called a **pivot** and arrange the array so that:

Elements smaller than pivot | Pivot | Elements greater than pivot

Then Quick Sort is recursively applied to the left and right portions.

## **Error table:**

| S.No | Error                                          | Correction                         |
|------|------------------------------------------------|------------------------------------|
| .    |                                                |                                    |
| 1    | Incorrect Python syntax, missing : or brackets | Check syntax, brackets and colons  |
| 2    | Incorrect low/high boundaries are passed       | Use low, pivot-1 and pivot+1, high |
| 3    | Function keeps calling itself                  | Add if low < high:                 |

# LAB-5

**1.Aim:** Write a Python program to implement a singly linked list with the below operations:

1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at a specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

**Program:**

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
class SinglyLinkedList:
```

```
    def __init__(self):
```

```
        self.head = None
```

```
    #1.Create linked list
```

```
    def create(self):
```

```
        n=int(input("Enter number of nodes: "))
```

```
        foriin range(n):
```

```
            data = int(input(f"Enter data for node {i + 1}: "))
```

```
        new_node = Node(data)
```

```
        ifself.head is None:
```

```
            self.head = new_node
```

```
else:
```

```
    temp = self.head
```

```
    while temp.next:
```

```
        temp = temp.next
```

```
    temp.next= new_node
```

```
print("Linkedlist created successfully.")
```

```
# 2. Insert at beginning
```

```
def insert_beginning(self):
```

```
    data=int(input("Enter data: "))
```

```
    new_node = Node(data)
```

```
    new_node.next= self.head
```

```
    self.head = new_node
```

```
    print("Nodeinserted at beginning.")
```

```
# 3. Insert at end
```

```
def insert_end(self):
```

```
    data=int(input("Enter data: "))
```

```
    new_node = Node(data)
```

```
    if self.head is None:
```

```
        self.head = new_node
```

```
    else:
```

```
        temp = self.head
```

```
while temp.next:
```

```
    temp = temp.next
```

```
    temp.next = new_node
```

```
print("Node inserted at end.")
```

```
#4.Insertat specific index
```

```
def insert_index(self):
```

```
    index=int(input("Enter index: "))
```

```
    data=int(input("Enter data: "))
```

```
    new_node = Node(data)
```

```
    if index== 0:
```

```
        new_node.next = self.head
```

```
        self.head = new_node
```

```
        print("Node inserted.")
```

```
        return
```

```
    temp = self.head
```

```
    for i in range(index - 1):
```

```
        if temp is None:
```

```
            print("Invalid index.")
```

```
            return
```

```
        temp= temp.next
```

```
    if temp is None:
```

```
        print("Invalid index.")
```

```
        return
```

```
new_node.next = temp.next
```

```
temp.next = new_node
```

```
print("Node inserted.")
```

#5.Delete by value

```
def delete_value(self):
```

```
    value = int(input("Enter value to delete: "))
```

```
    if self.head is None:
```

```
        print("List is empty.")
```

```
        return
```

```
    if self.head.data == value:
```

```
        self.head = self.head.next
```

```
        print("Node deleted.")
```

```
        return
```

```
    temp = self.head
```

```
    while temp.next and temp.next.data != value:
```

```
        temp = temp.next
```

```
    if temp.next is None:
```

```
        print("Value not found.")
```

```
    else:
```

```
        temp.next = temp.next.next
```

```
        print("Node deleted.")
```

#6.Delete first node

```
def delete_first(self):
```

```
ifself.head is None:
```

```
    print("List is empty.")
```

```
else:
```

```
    self.head = self.head.next
```

```
    print("First node deleted.")
```

```
#7.Delete last node
```

```
def delete_last(self):
```

```
    ifself.head is None:
```

```
        print("List is empty.")
```

```
        return
```

```
    ifself.head.next is None:
```

```
        self.head = None
```

```
        print("Last node deleted.")
```

```
        return
```

```
    temp = self.head
```

```
    while temp.next.next:
```

```
        temp = temp.next
```

```
    temp.next = None
```

```
    print("Last node deleted.")
```

```
#8.Count nodes
```

```
def count(self):
```

```
    count = 0
```

```
    temp = self.head
```

```
    while temp:
```

```
count += 1
```

```
temp = temp.next
```

```
print("Number of nodes:", count)
```

```
# 9. Display / Traverse
```

```
def display(self):
```

```
    if self.head is None:
```

```
        print("List is empty.")
```

```
        return
```

```
temp = self.head
```

```
while temp:
```

```
    print(temp.data, end=" -> ")
```

```
    temp = temp.next
```

```
print("None")
```

```
# Main program
```

```
sll = SinglyLinkedList()
```

```
while True:
```

```
    print("\n--- SINGLY LINKED LIST ---")
```

```
    print("1. Create linked list")
```

```
    print("2. Insert at beginning")
```

```
    print("3. Insert at end")
```

```
    print("4. Insert at specific index")
```

```
    print("5. Delete by value")
```

```
    print("6. Delete first node")
```

```
    print("7. Delete last node")
```

```
print("8. Count number of nodes")

print("9. Display / Traverse")

print("10. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:

    sll.create()

elif choice == 2:

    sll.insert_beginning()

elif choice == 3:

    sll.insert_end()

elif choice == 4:

    sll.insert_index()

elif choice == 5:

    sll.delete_value()

elif choice == 6:

    sll.delete_first()

elif choice == 7:

    sll.delete_last()

elif choice == 8:

    sll.count()

elif choice == 9:

    sll.display()

elif choice == 10:

    print("Program exited.")
```

break

else:

print("Invalid choice.")

## OUTPUT:

===== RESTART: D:/DSA Programs/programs/Week5a.py =====

----- SINGLY LINKED LIST -----

```
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 14
Enter data: 23
Enter data: 56
Linked List created successfully
```

----- SINGLY LINKED LIST -----

```
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 27
```

----- SINGLY LINKED LIST -----

```
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 4
```

---

```
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 4
```

----- SINGLY LINKED LIST -----

```
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
```

- 7. Delete from Index
- 8. Count Number of nodes
- 9. Display
- 10. Exit

Enter your choice: 5

----- SINGLY LINKED LIST -----

- 1. Create Linked List
  - 2. Insert at Beginning
  - 3. Insert at End
  - 4. Insert at Index
  - 5. Delete from Beginning
  - 6. Delete from End
  - 7. Delete from Index
  - 8. Count Number of nodes
  - 9. Display
  - 10. Exit
- Enter your choice: 6

----- SINGLY LINKED LIST -----

- 1. Create Linked List
- 2. Insert at Beginning
- 3. Insert at End
- 4. Insert at Index
- 5. Delete from Beginning
- 6. Delete from End
- 7. Delete from Index
- 8. Count Number of nodes
- 9. Display
- 10. Exit

Enter your choice: 7

Enter index: 6

Index out of bounds!

----- SINGLY LINKED LIST -----

- 1. Create Linked List
- 2. Insert at Beginning
- 3. Insert at End
- 4. Insert at Index
- 5. Delete from Beginning
- 6. Delete from End
- 7. Delete from Index
- 8. Count Number of nodes
- 9. Display
- 10. Exit

Enter your choice: 8

Number of nodes: 4

----- SINGLY LINKED LIST -----

- 1. Create Linked List
- 2. Insert at Beginning
- 3. Insert at End

```
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
14 -> 23 -> 56 -> 4 -> None
```

```
----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 10
```

## **Explanation:**

### Singly Linked List – Explanation

A **singly linked list** is a linear data structure in which elements are stored in parts:

**nodes.** Each node contains two

1. **Data** – stores the value.
2. **Next** – stores the address/reference of the next node.
  - self.data stores the node value.
  - self.next stores the reference to the next node.
  - Initially, next = None because the node is not connected to another node.

**Error table:**

| S.No | Mistake                                                                    | Correction                                           |
|------|----------------------------------------------------------------------------|------------------------------------------------------|
| 1    | <code>Node</code> class is not defined before using it                     | Define the <code>Node</code> class first             |
| 2    | <code>temp</code> is not updated inside the loop                           | Add <code>temp = temp.next</code>                    |
| 3    | <code>head</code> is not updated                                           | Use <code>self.head = new_node</code>                |
| 4    | User enters an index outside the list range                                | Validate the index before insertion                  |
| 5    | The given value does not exist                                             | Search the entire list and display "Value not found" |
| 6    | <code>head</code> is <code>None</code> and operation is performed directly | Check if <code>self.head</code> is <code>None</code> |

**2.Aim:** Write a Python program to implement a Double linked list with the above operations

**Program:**

class Node:

def \_\_init\_\_(self, data):

self.data = data

self.prev = None

self.next = None

```
class DoublyLinkedList:
```

```
    def __init__(self):
```

```
        self.head = None
```

```
# 1. Create linked list
```

```
def create(self):
```

```
    n = int(input("Enter number of nodes: "))
```

```
    for i in range(n):
```

```
        data = int(input(f"Enter data for node {i + 1}: "))
```

```
        new_node = Node(data)
```

```
        if self.head is None:
```

```
            self.head = new_node
```

```
        else:
```

```
            temp = self.head
```

```
            while temp.next:
```

```
                temp = temp.next
```

```
            temp.next = new_node
```

```
            new_node.prev = temp
```

```
    print("Doubly linked list created successfully.")
```

```
# 2. Insert at beginning
```

```
def insert_beginning(self):
```

```
    data = int(input("Enter data: "))
```

```
    new_node = Node(data)
```

```
new_node = Node(data)

if self.head is not None:

    new_node.next = self.head

    self.head.prev = new_node

self.head = new_node

print("Node inserted at beginning.")
```

# 3. Insert at end

```
def insert_end(self):

    data=int(input("Enter data: "))

    new_node = Node(data)

    if self.head is None:

        self.head = new_node

    else:

        temp = self.head

        while temp.next:

            temp = temp.next

        temp.next = new_node

        new_node.prev = temp

    print("Node inserted at end.")
```

#4. Insert at specific index

```
def insert_index(self):

    index=int(input("Enter index: "))

    data=int(input("Enter data: "))
```

```
new_node = Node(data)

if index == 0:

    if self.head:

        new_node.next = self.head

        self.head.prev = new_node

    self.head = new_node

    print("Node inserted.")

    return

temp = self.head

for i in range(index - 1):

    if temp is None:

        print("Invalid index.")

        return

    temp = temp.next

if temp is None:

    print("Invalid index.")

    return

new_node.next = temp.next

new_node.prev = temp

if temp.next:

    temp.next.prev = new_node

temp.next = new_node

print("Node inserted.")
```

#### # 5. Delete by value

```
def delete_value(self):  
  
    value = int(input("Enter value to delete: "))  
  
    temp = self.head  
  
    while temp and temp.data != value:  
  
        temp = temp.next  
  
    if temp is None:  
  
        print("Value not found.")  
  
        return  
  
    if temp.prev:  
  
        temp.prev.next = temp.next  
  
    else:  
  
        self.head = temp.next  
  
    if temp.next:  
  
        temp.next.prev = temp.prev  
  
    print("Node deleted.")
```

#### # 6. Delete first node

```
def delete_first(self):  
  
    if self.head is None:  
  
        print("List is empty.")  
  
        return  
  
    self.head = self.head.next  
  
    if self.head:  
  
        self.head.prev = None
```

```
print("First node deleted.")
```

```
#7.Delete last node
```

```
def delete_last(self):
```

```
    if self.head is None:
```

```
        print("List is empty.")
```

```
        return
```

```
    temp = self.head
```

```
    while temp.next:
```

```
        temp = temp.next
```

```
    if temp.prev:
```

```
        temp.prev.next = None
```

```
    else:
```

```
        self.head = None
```

```
    print("Last node deleted.")
```

```
#8.Count nodes
```

```
def count(self):
```

```
    count = 0
```

```
    temp = self.head
```

```
    while temp:
```

```
        count += 1
```

```
        temp = temp.next
```

```
    print("Number of nodes:", count)
```

```
#9.Display / Traverse
```

```
def display(self):
```

```
if self.head is None:
```

```
    print("List is empty.")
```

```
    return
```

```
temp = self.head
```

```
while temp:
```

```
    print(temp.data, end=" <-> ")
```

```
    temp = temp.next
```

```
print("None")
```

```
# Main program
```

```
dll = DoublyLinkedList()
```

```
while True:
```

```
    print("\n--- DOUBLY LINKED LIST ---")
```

```
    print("1. Create linked list")
```

```
    print("2. Insert at beginning")
```

```
    print("3. Insert at end")
```

```
    print("4. Insert at specific index")
```

```
    print("5. Delete by value")
```

```
    print("6. Delete first node")
```

```
    print("7. Delete last node")
```

```
print("8. Count number of nodes")
```

```
    print("9. Display / Traverse")
```

```
    print("10. Exit")
```

```
choice = int(input("Enter your choice: "))
```

```
ifchoice == 1:

    dll.create()

elifchoice == 2:

    dll.insert_beginning()

elifchoice == 3:

    dll.insert_end()

elifchoice == 4:

    dll.insert_index()

elifchoice == 5:

    dll.delete_value()

elifchoice == 6:

    dll.delete_first()

elifchoice == 7:

    dll.delete_last()

elifchoice == 8:

    dll.count()

elifchoice == 9:

    dll.display()

elifchoice == 10:

    print("Program exited.")

    break

else:

    print("Invalid choice.")
```

## Output:

```
===== RESTART: D:/DSA Programs/programs/Week5b.py =====

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 4
Enter data: 6
Enter data: 8
Enter data: 8
Enter data: 3
Linked List created successfully

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 5

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 65

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 4
Enter index: 3
Enter data: 90

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
```

6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit

Enter your choice: 5

----- DOUBLY LINKED LIST -----

1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit

Enter your choice: 6

----- DOUBLY LINKED LIST -----

1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit

Enter your choice: 7

Enter index: 4

----- DOUBLY LINKED LIST -----

1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit

Enter your choice: 8

Number of nodes: 4

----- DOUBLY LINKED LIST -----

1. Create Linked List
2. Insert at Beginning
3. Insert at End

```
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
6 -> 8 -> 90 -> 8 -> None
```

```
----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 10
```

### **Explanation:**

A **Doubly Linked List** is a linear data structure in which each node contains **three parts**:

1. prev – reference to the previous node
2. data – stores the value
3. next – reference to the next node

head points to the first node.

The first node's prev is None.

The last node's next is None.

Each node can be traversed **forward and backward**.

| S.No | Error                                       | Correction                           |
|------|---------------------------------------------|--------------------------------------|
| 1    | Trying to access prev of a nonexistent node | Use an appropriate if condition      |
| 2    | prev links are not updated                  | Correctly update prev references     |
| 3    | head becomes None                           | Check whether the list becomes empty |
| 4    | Second-last node's next is not changed      | Set temp.prev.next = None            |
| 5    | Index is outside the list                   | Validate the index                   |

**2.Aim:** Write a Python program to implement a Circular linked list with the above operations

**Program:**

class Node:

def \_\_init\_\_(self, data):

self.data = data

self.next = None

class CircularLinkedList:

def \_\_init\_\_(self):

self.head = None

## # 1. Create linked list

```
def create(self):

    n=int(input("Enter number of nodes: "))

    for i in range(n):

        data=int(input(f'Enter data for node {i + 1}: '))

        new_node = Node(data)

        if self.head is None:

            self.head = new_node

            new_node.next = self.head

        else:

            temp = self.head

            while temp.next != self.head:

                temp = temp.next

            temp.next = new_node

            new_node.next = self.head

    print("Circularlinked list created successfully.")
```

## # 2. Insert at beginning

```
def insert_beginning(self):

    data=int(input("Enter data: "))

    new_node = Node(data)

    if self.head is None:

        self.head = new_node

        new_node.next= self.head
```

else:

temp = self.head

while temp.next != self.head:

temp = temp.next

new\_node.next = self.head

temp.next = new\_node

self.head = new\_node

print("Node inserted at beginning.")

#3.Insert at end

def insert\_end(self):

data= int(input("Enter data: "))

new\_node = Node(data)

if self.head is None:

self.head = new\_node

new\_node.next = self.head

else:

temp = self.head

while temp.next != self.head:

temp = temp.next

temp.next = new\_node

new\_node.next = self.head

print("Node inserted at end.")

#4.Insertat specific index

```
def insert_index(self):

    index=int(input("Enter index: "))

    data=int(input("Enter data: "))

    new_node = Node(data)

    if index== 0:

        if self.head is None:

            self.head = new_node

            new_node.next = self.head

        else:

            temp = self.head

            while temp.next != self.head:

                temp = temp.next

            new_node.next = self.head

            temp.next = new_node

            self.head = new_node

        print("Node inserted.")

        return

    if self.head is None:

        print("Invalid index.")

        return

    temp = self.head

    for i in range(index - 1):

        temp= temp.next
```

# 5. Delete by value

```
def delete_value(self):
```

```
    value=int(input("Enter value to delete: "))
```

```
    if self.head is None:
```

```
        print("List is empty.")
```

```
        return
```

```
    current = self.head
```

```
    previous = None
```

```
    while True:
```

```
        if current.data== value:
```

```
            break
```

```
        previous = current
```

```
        current = current.next
```

```
    if current == self.head:
```

```
        print("Value not found.")
```

```
        return
```

```
# Only one node
```

```
if current==self.head and current.next == self.head:
```

```
    self.head = None
```

```
# Delete head
```

```
elif current == self.head:
```

```
    last = self.head
```

```
    while last.next!= self.head:
```

```
        last = last.next
```

```
        self.head = self.head.next

        last.next = self.head

#Delete other node

else:

    previous.next = current.next

    print("Node deleted.")

#6.Delete first node

def delete_first(self):

    if self.head is None:

        print("List is empty.")

        return

    #Only one node

    if self.head.next == self.head:

        self.head = None

    else:

        last = self.head

        while last.next != self.head:

            last = last.next

        self.head = self.head.next

        last.next = self.head

    print("First node deleted.")

#7.Delete last node

def delete_last(self):

    if self.head is None:
```

```
    print("List is empty.")

    return

# Only one node

if self.head.next == self.head:

    self.head = None

    print("Last node deleted.")

    return

temp = self.head

while temp.next.next != self.head:

    temp = temp.next

temp.next = self.head

print("Last node deleted.")

# 8. Count nodes

def count(self):

    if self.head is None:

        print("Numberofnodes: 0")

        return

    count = 0

    temp = self.head

    while True:

        count += 1

        temp = temp.next

        if temp == self.head:

            break

    print("Numberofnodes:", count)
```

## # 9. Display / Traverse

```
def display(self):  
  
    if self.head is None:  
  
        print("List is empty.")  
  
        return  
  
    temp = self.head  
  
    while True:  
  
        print(temp.data, end=" -> ")  
  
        temp = temp.next  
  
        if temp == self.head:  
  
            break  
  
    print("(back to head)")
```

## # Main program

```
c11 = CircularLinkedList()  
  
while True:  
  
    print("\n--- CIRCULAR LINKED LIST ---")  
  
    print("1. Create linked list")  
  
    print("2. Insert at beginning")  
  
    print("3. Insert at end")  
  
    print("4. Insert at specific index")  
  
    print("5. Delete by value")  
  
    print("6. Delete first node")  
  
    print("7. Delete last node")  
  
    print("8. Count number of nodes")
```

```
print("9. Display / Traverse")

print("10. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:

    cll.create()

elif choice == 2:

    cll.insert_beginning()

elif choice == 3:

    cll.insert_end()

elif choice == 4:

    cll.insert_index()

elif choice == 5:

    cll.delete_value()

elif choice == 6:

    cll.delete_first()

elif choice == 7:

    cll.delete_last()

elif choice == 8:

    cll.count()

elif choice == 9:

    cll.display()

elif choice == 10:

    print("Program exited.")

    break

else:

    print("Invalid choice.")
```

## Output:

```
===== RESTART: D:/DSA Programs/programs/Week5c.py =====

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 21
Enter data: 78
Enter data: 11
Linked List created successfully

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 4

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 8

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 4
Enter index: 2
Enter data: 54

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
```

5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit

Enter your choice: 5

----- CIRCULAR LINKED LIST -----

1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit

Enter your choice: 6

----- CIRCULAR LINKED LIST -----

1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit

Enter your choice: 7

Enter index: 2

----- CIRCULAR LINKED LIST -----

1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit

Enter your choice: 8

Number of nodes: 3

----- CIRCULAR LINKED LIST -----

1. Create Linked List
2. Insert at Beginning

```
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
21 -> 54 -> 11 -> (head)
```

```
----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 10
|
```

### **Explanation:**

A Circular Linked List is a linked list in which the **last node points back to the first node** instead of pointing to None.

New node points to the old head.

Last node points to the new head.

## Error table:

| S.No | Error                                 | Correction                                             |
|------|---------------------------------------|--------------------------------------------------------|
| 1    | Infinite loop during traversal        | Stop when <code>temp == head</code>                    |
| 2    | Circular connection is lost           | Set last node's <code>next</code> to <code>head</code> |
| 3    | Insert at beginning breaks the circle | Update <code>last.next = new_node</code>               |
| 4    | Count is incorrect                    | Stop when pointer reaches <code>head</code>            |
| 5    | Invalid index                         | Check index before insertion                           |

# LAB-6

**1.Aim:** Write a Python program to implement stack operations such as **Push, Pop, Peek, Display** using an array

**Program:**

# Stack implementation using Array

```
stack = [543,45,324,435,34,45,32]
```

```
MAX_SIZE = 5
```

```
def push():
```

```
    if len(stack) == MAX_SIZE:
```

```
        print("Stack Overflow")
```

```
    else:
```

```
        element = int(input("Enter element to push: "))
```

```
        stack.append(element)
```

```
    print(element, "pushed into stack")
```

```
def pop():
```

```
    if len(stack) == 0:
```

```
        print("Stack Underflow")
```

```
    else:
```

```
        element = stack.pop()
```

```
    print(element, "popped from stack")
```

```
def peek():
```

```
    if len(stack) == 0:
```

```
        print("Stack is empty")
```

else:

```
print("Top element is:", stack[-1])
```

def display():

if len(stack) == 0:

```
print("Stack is empty")
```

else:

```
print("Stack elements are:")
```

```
for i in range(len(stack) - 1, -1, -1):
```

```
print(stack[i])
```

Menu-driven program

while True:

```
print("\n--- STACK USING ARRAY ---")
```

```
print("1. Push")
```

```
print("2. Pop")
```

```
print("3. Peek")
```

```
print("4. Display")
```

```
print("5. Exit")
```

```
choice = int(input("Enter your choice: "))
```

if choice == 1:

```
push()
```

elif choice == 2:

```
pop()
```

elif choice == 3:

```
peek()
```

```

elif choice == 4:

    display()

elif choice == 5:

    print("Program terminated")

    break

else:

    print("Invalid choice")

```

## Output:

```

===== RESTART: D:/DSA Programs/programs/Week6a.py =====
Enter the size of the stack: 4

----- STACK OPERATIONS -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to push: 5
5 pushed onto the stack.

----- STACK OPERATIONS -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to push: 6
6 pushed onto the stack.

----- STACK OPERATIONS -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
Popped element: 6

----- STACK OPERATIONS -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element is: 5

----- STACK OPERATIONS -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (Top to Bottom):
| 5 |
-----

```

### **Explanation:**

A **stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle. The element inserted last is removed first.

The main stack operations are:

1. **Push** – Adds an element to the top of the stack.
2. **Pop** – Removes the top element from the stack.
3. **Peek** – Displays the top element without removing it.
4. **Display** – Displays all elements in the stack.

### **Error table:**

| S.No | Error                               | Correction                                    |
|------|-------------------------------------|-----------------------------------------------|
| 1    | IndexError: pop from empty list     | Check <code>len(stack) == 0</code> before pop |
| 2    | Stack Overflow                      | Check <code>len(stack) == MAX_SIZE</code>     |
| 3    | IndexError: list index out of range | Check whether stack is empty before Peek      |
| 4    | ValueError                          | Enter a valid integer                         |

**2.Aim:** Write a Python program to implement stack operations such as **Push, Pop, Peek, Display** using a **linked list**.

### **Program:**

#Stack implementation using Linked List

```
class Node:
```

```
    def init(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
class Stack:
```

```
    def init(self):
```

```
        self.top = None
```

```
    # Push operation
```

```
    def push(self):
```

```
        element = int(input("Enter element to push: "))
```

```
        new_node = Node(element)
```

```
        new_node.next = self.top
```

```
        self.top = new_node
```

```
        print(element, "pushed into stack")
```

```
    # Pop operation
```

```
    def pop(self):
```

```
        if self.top is None:
```

```
            print("Stack Underflow")
```

```
        else:
```

```
            element = self.top.data
```

```
            self.top = self.top.next
```

```
            print(element, "popped from stack")
```

# Peek operation

```
def peek(self):  
    if self.top is None:  
        print("Stack is empty")  
    else:  
        print("Top element is:", self.top.data)
```

# Display operation

```
def display(self):  
    if self.top is None:  
        print("Stack is empty")  
    else:  
        temp = self.top  
        print("Stack elements are:")  
  
        while temp is not None:  
            print(temp.data)  
            temp = temp.next
```

#Menu-driven program

```
stack = Stack()
```

```
while True:
```

```
    print("\n--- STACK USING LINKED LIST ---")
```

```
    print("1. Push")
```

```
    print("2. Pop")
```

```
    print("3. Peek")
```

```
    print("4. Display")
```

```
    print("5. Exit")
```

```

choice = int(input("Enter your choice: "))
if choice == 1:
    stack.push()
elif choice == 2:
    stack.pop()
elif choice == 3:
    stack.peak()
elif choice == 4:
    stack.display()
elif choice == 5:
    print("Program terminated")
    break
else:
    print("Invalid choice")

```

## Output:

```

= RESTART: C:/Users/Manaswini/AppData/Local/Programs/Python/Python314/Week6b.py
manaswini-cse25102

----- STACK OPERATIONS (LINKED LIST) -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to push: 3
3 pushed onto the stack.

----- STACK OPERATIONS (LINKED LIST) -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to push: 6
6 pushed onto the stack.

----- STACK OPERATIONS (LINKED LIST) -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter data to push: 8
8 pushed onto the stack.

----- STACK OPERATIONS (LINKED LIST) -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2

```

```

----- STACK OPERATIONS (LINKED LIST) -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (Top to Bottom):
| 6 |
| 3 |
-----

----- STACK OPERATIONS (LINKED LIST) -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting stack program.

```

## Explanation:

A linked-list stack consists of **nodes**.

A temporary pointer travels through the linked list:

## Error table:

| S.No | Error           | Correction                                   |
|------|-----------------|----------------------------------------------|
| 1    | AttributeError  | Check self.top is None before accessing data |
| 2    | Stack Underflow | Check whether top is None                    |
| 4    | NameError       | Use the correct variable/reference           |

# Lab-7

**1.Aim:** Write a Python program to implement a **queue** and its operations such as **Enqueue, Dequeue, Peek, Display** with a **menu-driven approach**. Implement using **array**

## Program:

```
#Queue Implementation using Linked List
```

```
#Operations: Enqueue, Dequeue, Peek, Display
```

```
class Node:
```

```
    def init(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
class Queue:
```

```
    def init(self):
```

```
        self.front = None
```

```
        self.rear = None
```

```
# Enqueue operation
```

```
def enqueue(self, data):
```

```
    new_node = Node(data)
```

```
    if self.rear is None:
```

```
        self.front = self.rear = new_node
```

```
    else:
```

```
        self.rear.next = new_node
```

```
        self.rear = new_node
```

```
    print(data, "inserted into queue.")
```

```
# Dequeue operation
def dequeue(self):
    if self.front is None:
        print("Queue Underflow! Queue is empty.")
        return
    value = self.front.data
    self.front = self.front.next
    if self.front is None:
        self.rear = None
    print(value, "deleted from queue.")

# Peek operation
def peek(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        print("Front element:", self.front.data)

# Display operation
def display(self):
    if self.front is None:
        print("Queue is empty.")
        return
    current = self.front
    print("Queue elements:", end=" ")
    while current is not None:
        print(current.data, end=" ")
        current = current.next
    print()

queue = Queue()

while True:

    print("\n----- QUEUE USING LINKED LIST -----")

    print("1. Enqueue")

    print("2. Dequeue")

    print("3. Peek")

    print("4. Display")

    print("5. Exit")
```

```
choice = int(input("Enter your choice: "))
if choice == 1:
    value = int(input("Enter element to enqueue: "))
    queue.enqueue(value)
elif choice == 2:
    queue.dequeue()

elif choice == 3:
    queue.peek()
elif choice == 4:
    queue.display()

elif choice == 5:
    print("Program terminated.")
    break
else:
    print("Invalid choice! Please try again.")
```

## Output:

```
= RESTART: C:/Users/Manaswini/AppData/Local/Programs/Python/Python314/Week7a.py
manaswini-cse25102

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 2
2 inserted into the queue.

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 4
4 inserted into the queue.

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
2 deleted from the queue.

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 4
```

```

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: 4

--- QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting program...

```

### **Explanation:**

A **queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle.

In this program, Python's **list** is used as an array to store queue elements.

### **Error table:**

| S.No | Error                                                                                                                      | Rectification                                                 |
|------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| 1    | Using <code>queue.pop()</code> instead of <code>queue.pop(0)</code> removes the last element instead of the front element. | Use <code>queue.pop(0)</code> for dequeue operation.          |
| 2    | Not checking the queue size before insertion can cause the queue to exceed its maximum size.                               | Check if <code>len(queue) == MAX_SIZE</code> before enqueue.  |
| 3    | Accessing <code>queue[0]</code> when the queue is empty causes an <code>IndexError</code> .                                | Check if <code>len(queue) == 0</code> before performing Peek. |

**2.Aim:** Write a Python program to implement a **queue** and its operations such as **Enqueue, Dequeue, Peek, Display** with a **menu-driven approach**. Implement using **linked-list data structures**.

**Program:**

~~#Queue Implementation using Linked List~~

#Operations: Enqueue, Dequeue, Peek, Display

class Node:

def **init**(self, data):

self.data = data

self.next = None

class Queue:

def **init**(self):

self.front = None

self.rear = None

```
# Enqueue operation
def enqueue(self, data):
    new_node = Node(data)
    if self.rear is None:
        self.front = self.rear = new_node
    else:
        self.rear.next = new_node
        self.rear = new_node
    print(data, "inserted into queue.")

# Dequeue operation
def dequeue(self):
    if self.front is None:
        print("Queue Underflow! Queue is empty.")
        return

    value = self.front.data
    self.front = self.front.next

    if self.front is None:
        self.rear = None
    print(value, "deleted from queue.")
```

```
# Peek operation
def peek(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        print("Front element:", self.front.data)

# Display operation
def display(self):
    if self.front is None:
        print("Queue is empty.")
        return
    current = self.front
    print("Queue elements:", end=" ")
    while current is not None:
        print(current.data, end=" ")
        current = current.next
    print()

queue = Queue()

while True:

    print("\n----- QUEUE USING LINKED LIST -----")

    print("1. Enqueue")

    print("2. Dequeue")

    print("3. Peek")

    print("4. Display")

    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter element to enqueue: "))
        queue.enqueue(value)
```

```
elif choice == 2:
    queue.dequeue()

elif choice == 3:
    queue.peek()

elif choice == 4:
    queue.display()

elif choice == 5:
    print("Program terminated.")
    break

else:
    print("Invalid choice! Please try again.")
```

## Output:

```
= RESTART: C:/Users/Manaswini/AppData/Local/Programs/Python/Python314/Week7b.py
manaswini-cse25102

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 2
2 inserted into the queue.

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 4
4 inserted into the queue.

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 6
6 inserted into the queue.

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
```

```

2 deleted from the queue.

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 4

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: 4 6

--- CIRCULAR QUEUE USING ARRAY ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting program...

```

## Explanation:

A **linked-list queue** consists of dynamically created nodes. Each node contains two parts:

Two pointers are maintained:

- **Front:** Points to the first node.
- **Rear:** Points to the last node.

## Error table:

| S.No | Error                                                                                   | Rectification                                       |
|------|-----------------------------------------------------------------------------------------|-----------------------------------------------------|
| 1    | After deleting the last node, rear may still point to the deleted node.                 | If front becomes None, set self.rear = None.        |
| 2    | Displaying the linked list without moving to the next node can create an infinite loop. | Use current = current.next inside the display loop. |